

sigc 관찰 프로브

실신포 특징 4줄 + find_bursts 단계 관찰 — [(0, n)] 의 이유를 현장에서 본다

코드 204줄 · 총 5쪽 | 2026-08-14 · 1662b89+수정중 | 의존성: 설치돼 있는 signus + numpy/scipy (matplotlib 이 있고 창을 띄울 수 있으면 그림 — ssh/무화면이면 자동으로 ASCII)

- 이 코드를 **signus/** 폴더 **옆에 sigc.py** 로 저장한다. 필사는 두 토막으로 끊어도 된다 — 「여기부터는 그림 단계」 주석 **앞**까지만 옮기면 ②③(요약 4줄) 이 다 돌고, 그 뒤는 ④의 그림 단계 전용이다.
- python3 sigc.py kat** — 아래 기대 4줄과 글자까지 비교. 다르면 스크립트 필사 오타다 (kat 이 잡아준다).
- python3 sigc.py <캡처파일>** — 요약 4줄을 **#코드까지** 그대로 받아쳐 회신한다. (파일명은 analyze 때처럼 fs·iq/real 을 실어야 한다)
- python3 sigc.py <캡처파일> 2** 처럼 단계 번호를 붙이면 그 단계를 그림/ASCII 로 보여준다 — 본 것을 말로 회신하면 된다.
- 받은 쪽(맥)은 **tools/sigc.py check** 로 먼저 오타 검증 + 출구 해석을 한다.

```
sigc a n66000 f1000000 ev86 ed55 sp26 dc0 cp4 cx1 #5zx2
sigc b c1030 g128 b155 cn156 t308 m520 dlo25 dhi45 #7hny
sigc c r7 dn97 gp90 sb33 av66 ah56 sk1 #vaw0
sigc d kb17 kc8 w7 p-14 pd56 ps59 q37 qd100 qs51 #7m4q
```

명령 인자	무엇을 보나	kat 에서 보여야 하는 것	실캡처에서 관찰·회신할 것
kat	자기검증	아래 기대 4줄과 글자까지 일치	매번 먼저. 다르면 값이 다른 줄부터 스크립트 재대조
(없음)	요약 4줄	kat 과 같은 형식	4줄을 #코드까지 그대로 받아쳐 회신
1	읽기 확인	n 66000 · 길이 0.066s · 형식 iq	n·fs 가 캡처 실제와 맞는가
2	광대역 포락선 (베타)	계단이 또렷하고 분리도 0.86	계단(#)과 골(빈칸)이 보이는가, 분리도가 0.12 를 넘는가
3	위터폴 절대전력	세로 버스트 줄 + 가로로 안 끊기는 띠 2개	끊이지 않는 가로 띠(연속 방사체)가 있는가, 버스트 줄 수
4	빈별 바닥 대비 비율	세기 일정한 연속 띠는 사라지고 버스트만	find_bursts 가 보는 그림 — 버스트가 여기서도 살아 있는가
5	열 점수 + 문턱	봉우리가 hi 위, base 1.55 ≈ c_noise 1.56	봉우리가 hi 선을 넘는가, base 가 c_noise 근처인가
6	후보 런 + 실제 답	원시 후보 7개 → find_bursts → [(64, 6080), (11904, 180	원시 후보가 병합·가드를 거쳐 몇 개로 남는가

요약 4줄 읽는 법: a=포락선(ev 분리도x100 · ed 듀티% · sp 피크비dB · dc% · cp 포화 지표%) = 캡처 자신의 상위 0.1% 레벨에 붙은 표본 비율, 깨끗하면 2~4·포화면 900+ · cx1=복소/cx0=실수) b=셀 점수 지형(c 열수 · g 빈수 · b base · cn · t 문턱 · m 최대 · dlo/dhi 는 base 로부터의 문턱 여유, x100 — dlo25 dhi45 가 아니면 그 상수를 잘못 옮긴 것) c=버스트 구조(r 런수 · dn 길이열 · gp 간격열 · sb 점수dB · av/ah 문턱 위 열% · sk 스파이크런) d=방사체(kb 점유빈 · kc 연속점유빈 · w 최대폭빈 · p/q 위치빈 · pd/qd 듀티% · ps/qs 절대dB, q999=부 방사체 없음). 한 줄이 100칸을 넘으면 ↵ 이어짐 — 붙여서 입력. **kat 이 검증하는 것은 요약 4줄 경로 뿐이다** — 단계 2~6 의 그림 코드(curve/image/pool)는 kat 이 실행하지 않으므로, 그림 단계에서 처음 크래시하면 그 세 함수의 필사부터 다시 본다.

```

1  """sigc.py - 실신호 특징 4줄 + find_bursts 단계 관찰 프로브 (장비 필사용, signus/ 옆에 저장).
2      python3 sigc.py kat                # 자기검증 - 표지의 기대 4줄과 비교
3      python3 sigc.py <캡처> [1-6]       # 요약 4줄(받아쳐 회신) / 단계 관찰(그림 또는 ASCII)
4  """
5  import sys
6
7  import numpy as np
8  from scipy.ndimage import uniform_filter1d
9  from scipy.signal import get_window, stft
10
11 from signus import dsp, sigio
12 from signus.cli import check_code
13
14
15 def otsu(v, bins=128):
16     h, ed = np.histogram(v, bins=bins)
17     c = (ed[:-1] + ed[1:]) / 2
18     w = np.cumsum(h).astype(float)
19     m = np.cumsum(h * c)
20     mb = m / np.where(w > 0, w, 1)
21     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
22     return float(c[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
23
24
25 def runs_of(mask):
26     d = np.diff(mask.astype(np.int8))
27     s = list(np.where(d == 1)[0] + 1)
28     e = list(np.where(d == -1)[0] + 1)
29     if mask[0]:
30         s.insert(0, 0)
31     if mask[-1]:
32         e.append(mask.size)
33     return list(zip(s, e, strict=True))
34
35
36 if len(sys.argv) < 2:
37     raise SystemExit("사용법: python3 sigc.py kat | python3 sigc.py <캡처파일> [단계 1-6]")
38 arg, step = sys.argv[1], int(sys.argv[2]) if len(sys.argv) > 2 else 0
39 if arg == "kat":
40     fs, t = 1e6, np.arange(66000.0)      # 66000: 마지막 버스트가 파일 끝까지 이어진다
41     rng = np.random.default_rng(7)       # 시드 고정 스트림 - numpy 가 재현을 보장한다
42     x0 = 0.01 * (rng.standard_normal(66000) + 1j * rng.standard_normal(66000))
43     x0 += 0.2 * np.exp(2j * np.pi * 0.29 * t)
44     x0 += 0.5 * np.exp(-2j * np.pi * 0.11 * t) * ((t // 6000).astype(int) % 2 == 0)
45     x0 += 0.0028 * np.exp(2j * np.pi * 0.41 * t)    # 문턱 경계에 걸친 성분들: 필사 오타가
46     x0[31000:35000] += 0.0088 * np.exp(2j * np.pi * 0.35 * t[:4000])    # 상수 하나만
47     x0[42400:47600] += 0.0075 * np.exp(2j * np.pi * -0.37 * t[:5200])    # 스쳐도 아래
48     x0[200:260] = 0.95                # 요약 4줄 어딘가가 바뀌게 만든다
49     x0[21000] += 8.0
50     x0[15000] += 2.0
51 else:
52     x0, meta = sigio.read(arg)
53     fs = meta.fs
54 cx = np.iscomplexobj(x0)
55 raw = np.abs(np.asarray([x0.real, x0.imag])).max(0) if cx else np.abs(x0)
56 cp = float((raw > 0.98 * np.percentile(raw, 99.9)).mean())    # 만점(±1)이 아니라 캡처 자신의
57 # 99.9퍼센타일 기준: float 캡처는 만점 정규화가 안 돼 ±1 기준이면 통짜 오경보다(실측 cp972)
58 x = dsp.analytic(x0)
59 n, pw = x.size, np.abs(x) ** 2
60 rms = float(np.sqrt(pw.mean()) + 1e-30)

```

```

61 dc = abs(complex(x.mean())) / rms # dc 는 블록 전에 재고, 그 뒤는 pipeline 과 똑같이
62 x = x - x.mean() # DC 블록한 신호로 켜다 - 안 그러면 dc 큰 캡처에서
63 pw = np.abs(x) ** 2 # 프로브와 analyze 가 정반대 진단을 낸다
64 # 러닝합 잔차 때문에 정확히 0 인 구간(스켈치/뮤트)에서 음수가 나와 log10 이 NaN 이 되고,
65 # otsu 의 histogram 이 장비에서 생 트레이스백으로 죽었다 - 0 으로 눌러 막는다.
66 lp = np.log10(np.maximum(uniform_filter1d(pw, max(64, n // 1000)), 0) + 1e-20)
67 hot = lp >= (et := otsu(lp))
68 ev = float(lp[hot].mean() - lp[~hot].mean()) if 0 < hot.sum() < n else 0.0
69 ed = float(hot.mean())
70 sp = 10 * np.log10(pw.max() / (pw.mean() + 1e-30) + 1e-30)
71 nper = int(min(128, max(64, 1 << int(np.log2(max(n // 2, 64)))))
72 hop = nper // 2
73 _, _, z = stft(x, fs=fs, window=get_window("blackmanharris", nper), nperseg=nper,
74             noverlap=nper - hop, return_onesided=False, boundary=None, padded=False)
75 P = np.abs(z) ** 2
76 nb, nc = P.shape
77 fl = np.percentile(P, 10, axis=1)
78 r = P / (fl[:, None] + 1e-30)
79 k = max(3, nb // 16)
80 sc = uniform_filter1d(np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30), 3)
81 thr = otsu(sc, bins=64)
82 quiet = sc[sc < thr]
83 base = float(np.median(quiet)) if quiet.size else 9.99
84 cn = float(np.log10((np.log(nb / k) + 1) / 0.105))
85 lov, hiv = base + 0.25, base + 0.45 # find_bursts 의 두 문턱. 각 상수는 한 곳에만 쓰고,
86 above, hi_m = sc >= lov, sc >= hiv # b 줄에 dlo/dhi 로 되찍는다 - 오타면 그 숫자가 바뀐다
87 rr = [(s, e) for s, e in runs_of(above) if hi_m[s:e].any()]
88 spans = [(max(0, s * hop) + (hop if e - s >= 6 else 0), # 6월 이상은 양끝 한 hop
89          min(n, (e - 1) * hop + nper) - (hop if e - s >= 6 else 0)) # 씹 자른다 - dsp 의
90          for s, e in rr] # 스파이크 게이트와 같은
91 segs = [pw[s:e] for s, e in spans] # 구간에서 재려고
92 sk = sum(float(sg.max() / (sg.mean() + 1e-30)) > 60.0 for sg in segs)
93 gp_ = np.array([rr[i + 1][0] - rr[i][1] for i in range(len(rr) - 1)])
94 sb = round(10 * float(np.median([sc[s:e].max() for s, e in rr]) - base)) if rr else 0
95 g = float(np.median(fl if cx else fl[:nb // 2]) + 1e-30) # real 은 해석신호라 음수 반쪽이
96 # 비어 있다: 전 빈 중앙값을 쓰면 바닥이 0 으로 내려가 밴드 절반이 '점유'로 읽힌다(실측)
97 Ps, fls = np.fft.fftshift(P, 0), np.fft.fftshift(fl)
98 du = (Ps > 40 * g).mean(1)
99 occ = du > 0.1
100 kb, kcont = int(occ.sum()), int((fls > 5 * g).sum())
101 grp = runs_of(occ) if occ.any() else []
102 wd = max((e - s for s, e in grp), default=0)
103 p95 = np.percentile(Ps, 95, axis=1) / g
104 pk = int(np.argmax(p95))
105 pgrp = next(((s, e) for s, e in grp if s <= pk < e), (pk, pk + 1))
106 m2 = np.where(occ, p95, 0.0)
107 m2[pgrp[0]:pgrp[1]] = 0.0
108 q2 = int(np.argmax(m2))
109 qo, qd, qs = (q2 - nb // 2, round(100 * float(du[q2])),
110             round(10 * np.log10(m2[q2] + 1e-30))) if m2[q2] > 0 else (999, 0, 0)
111 la = (f"sigc a n{n} f{fs:.0f} ev{round(100 * ev)} ed{round(100 * ed)}"
112      f" sp{round(float(sp))} dc{round(100 * dc)} cp{round(1000 * cp)}"
113      f" cx{int(cx)}")
114 lb = (f"sigc b c{nc} g{nb} b{round(100 * base)} cn{round(100 * cn)}"
115      f" t{round(100 * thr)} m{round(100 * float(sc.max()))}"
116      f" dlo{round(100 * (lov - base))} dhi{round(100 * (hiv - base))}")
117 lc = (f"sigc c r{len(rr)} dn{round(float(np.median([e - s for s, e in rr])))} if rr else 0}"
118      f" gp{round(float(np.median(gp_))} if gp_.size else 0} sb{sb}"
119      f" av{round(100 * float(above.mean()))} ah{round(100 * float(hi_m.mean()))}"
120      f" sk{sk}")

```

```

121 ld = (f"sigc d kb{kb} kc{kcont} w{wd} p{pk - nb // 2} pd{round(100 * float(du[pk]))}"
122         f" ps{round(10 * np.log10(p95[pk] + 1e-30))} q{qo} qd{qd} qs{qs}")
123 if step == 0:                                     # 요약 4줄 - 이것만 받아쳐서 회신한다
124     for s in (la, lb, lc, ld):
125         print(f"{s} #{check_code(s)}")
126     sys.exit()
127
128 # — 여기부터는 그림 단계(1~6) 전용이다. 요약 4줄만 쓸 거면 여기서 멈춰도 된다 —
129
130 try:
131     import matplotlib.pyplot as plt
132     plt = None if plt.get_backend().lower() == "agg" else plt    # 무화면(ssh)이면 ASCII 로
133 except ImportError:
134     plt = None
135
136
137 def pool(a, m):
138     a = np.asarray(a, float)
139     k = max(1, a.shape[-1] // m)
140     return a[..., :(a.shape[-1] // k) * k].reshape(*a.shape[:-1], -1, k).max(-1)
141
142
143 def curve(y, marks):
144     if plt:
145         plt.plot(y)
146         for la, v in marks:
147             plt.axhline(v, ls="--", label=f"{la} {v:.2f}")
148         plt.legend()
149         return plt.show()
150     # 얼마나 최소·최대를 같이 그린다: max 만 그리면 버스트 사이의 골이 지워져 계단이 통짜
151     # 블록으로 보이고, 관측자가 "평평하다(=베토)" 라고 정반대로 회신하게 된다.
152     kk = max(1, y.size // 100)
153     y2 = y[:y.size // kk * kk].reshape(-1, kk)
154     ylo, yhi = y2.min(1), y2.max(1)
155     bot = float(min(ylo.min(), *(v for _, v in marks)))
156     st = (float(max(yhi.max(), *(v for _, v in marks))) - bot) / 18 or 1.0
157     rl, rh = np.round((ylo - bot) / st).astype(int), np.round((yhi - bot) / st).astype(int)
158     rm = {}
159     for la, v in marks:
160         rm.setdefault(round((v - bot) / st), []).append(f"{la} {v:.2f}")
161     for r in range(18, -1, -1):
162         bg = "-" if r in rm else " "                # 문턱은 가로 전체 선으로 (겹치면 라벨 합침)
163         print("".join("#" if rl[i] >= r else ("+" if rh[i] >= r else bg)
164                     for i in range(rl.size)) + "|" + " / ".join(rm.get(r, [])))
165     print("#=이 구간 내내 그 위 · +=봉우리만 그 위 · -=문턱선")
166
167
168 def image(img):
169     if plt:
170         plt.imshow(img, aspect="auto", origin="lower", interpolation="nearest")
171         return plt.show()
172     sm = pool(pool(img, 100).T, 32).T
173     hi = sm.max() + 1e-30
174     lo = max(float(np.percentile(sm, 5)), hi - 4)    # 표시폭 4 decades 고정 - real 캡처는 빈
175     # 음수 반쪽이 1e-16 이하 5퍼센타일을 쓰면 눈금이 14 decades 로 늘어나 통짜 벽이 된다
176     for row in np.clip((sm - lo) / (hi - lo) * 9.99, 0, 9).astype(int)[::-1]:
177         print("".join(" .:~+*#%@"[i] for i in row))
178
179
180 if step == 1:

```

```

181     print(f"\n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 형식 {'iq' if np.iscomplexobj(x0) else 'real'}")
182     print(f"rms {rms:.4f} dc {100 * dc:.1f}% 클리핑 {1000 * cp:.1f}% 피크비 {float(sp):.0f}dB")
183     elif step == 2:
184         curve(lp, [("otsu", et)])
185         print(f"포락선 분리도 {ev:.3f} decades - 0.12 미만이면 find_bursts 는 통짜 [(0,n)] (베토)")
186         print(f"포락선 듀티 {100 * ed:.0f}% (버스트가 계단으로 보여야 정상)")
187     elif step == 3:
188         image(np.log10(Ps + 1e-20))
189         print("위터폴 절대전력 (아래=-fs/2, 위=+fs/2) - 가로로 끊기지 않는 띠 = 연속 방사체")
190     elif step == 4:
191         image(np.log10(np.fft.fftshift(r, 0) + 1e-30))
192         print("빈별 바닥 대비 비율 - find_bursts 가 보는 그림. 세기가 일정한 연속 방사체는 여기서"
193               " 사라지지만, 세기가 흔들리면 가짜 버스트로 남는다 (d 줄 pd 로 확인)")
194     elif step == 5:
195         curve(sc, [("base", base), ("hi", hiv), ("c_noise", cn)])
196         print(f"열 점수 - hi 위 봉우리가 버스트 후보. 런 {len(rr)}개, base {base:.2f}, cn {cn:.2f}")
197     elif step == 6:
198         for (s, e), (a0, a1), sg in zip(rr, spans, segs, strict=True):
199             print(f"열 {s}-{e} 샘플 {a0}-{a1} 높이 {10 * (float(sc[s:e].max()) - base):.0f}dB"
200                   f" 피크/평균 {float(sg.max() / (sg.mean() + 1e-30)):.0f} (60 초과=스파이크 탈락)")
201         print(f"위는 병합/가드 전 원시 후보 {len(rr)}개 - 아래가 실제로 분석에 쓰이는 답이다:")
202         fb = dsp.find_bursts(x, fs)
203         print(f"find_bursts → {fb[:6]}{'...' if len(fb) > 6 else ''}"
204               + (" ※ 통짜 = 버스트 미검출" if fb == [(0, n)] else f" 버스트 {len(fb)}개"))

```